

Experiment No.	01
Title	Design Static Webpage using HTML5 Components
Course Outcome	CO1

1. Objective / Aim

To design and implement a static webpage using HTML5 components — semantic elements, tables, and forms — demonstrating clear page structure and content organisation without the use of any CSS styling.

2. System Requirements

Hardware

Processor	Intel Core i5 or equivalent
RAM	8 GB or above
Storage	256 GB SSD / HDD

Software

Operating System	Windows 11 Home
Code Editor	Visual Studio Code with Live Server extension
Browser	Google Chrome / Microsoft Edge
Runtime	None — static HTML file only

3. Architecture / Design

File Structure

```
exp01/  
└─ index.html  <-- HTML5 structure (no CSS)
```

Page Layout (HTML5 Structure)

- <header> — logo (h1) laid out in a structural <table> with a right-aligned <nav>
- <section id="home"> — centered intro heading and paragraph
- Structural <table> (70% / 30% split)
 - → <main id="articles"> — two <article> blocks describing structural tags and interactive elements
 - → <form id="contact"> inside a <fieldset>/<legend> — text, email, select, textarea, submit & reset inputs
 - → <aside id="about"> — benefits list () and required components list ()
- <footer> — centered copyright text

No external or internal CSS is used anywhere on the page; every visual grouping (columns, spacing, alignment) is achieved purely through native HTML elements such as <table>, <center>, and <hr>.

4. Source Code — index.html

```
<!DOCTYPE html>  
<html lang="en">
```

```
<head>
  <meta charset="UTF-8">
  <title>Pure Semantic HTML Static Webpage</title>
</head>
<body>

  <!-- Header Section -->
  <header>
    <table width="100%" border="0" cellspacing="0" cellpadding="10">
      <tr>
        <td>
          <h1>StaticWeb</h1>
        </td>
        <td align="right">
          <nav>
            <a href="#home">Home</a> &nbsp;|&nbsp;&nbsp;&nbsp;
            <a href="#articles">Articles</a> &nbsp;|&nbsp;&nbsp;&nbsp;
            <a href="#about">About</a> &nbsp;|&nbsp;&nbsp;&nbsp;
            <a href="#contact">Contact Us</a>
          </nav>
        </td>
      </tr>
    </table>
  </header>

  <hr>

  <!-- Hero / Showcase Section -->
  <section id="home">
    <center>
      <h2>Semantic Structures Without CSS</h2>
      <p>This layout uses raw HTML components to build an organized document hierarchy that browsers can cleanly parse out-of-the-box.</p>
      <p><a href="#articles"><b>Read Our Articles Below &darr;</b></a></p>
    </center>
  </section>

  <hr>

  <!-- Main Content Layout using a Structural Table -->
  <table width="100%" border="0" cellspacing="0" cellpadding="20">
    <tr valign="top">

      <!-- Left Side: Main Content Column -->
      <td width="70%">
        <main id="articles">

          <article>
            <h2>1. Core Structural Tags</h2>
            <p>Published: July 7, 2026</p>
            <p>When you omit cascading style sheets entirely, the semantic markup handles all of the structural heavy lifting. Elements like <code>&lt;main&gt;</code>, <code>&lt;article&gt;</code>, and <code>&lt;section&gt;</code> tell screen readers and web scrapers exactly how information flows.</p>
            <p>Using raw browser defaults ensures excellent loading performance, perfect accessibility compliance, and absolute structural clarity.</p>
          </article>

          <br><hr><br>

          <article>
            <h2>2. Native Interaction Elements</h2>
            <p>Published: July 5, 2026</p>
            <p>Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.</p>
```

```
<h3>Interactive Demonstration Forms</h3>
<form id="contact" action="#" method="post">
  <fieldset>
    <legend><b>User Inquiry Form</b></legend>
    <p>
      <label for="name">Full Name: </label>
      <input type="text" id="name" name="username" required
placeholder="John Doe">
    </p>
    <p>
      <label for="email">Email Address: </label>
      <input type="email" id="email" name="useremail"
required>
    </p>
    <p>
      <label for="topic">Inquiry Topic: </label>
      <select id="topic" name="usertopic">
        <option value="general">General Feedback</option>
        <option value="technical">Technical
Architecture</option>
Curriculum</option>
        </select>
      </p>
      <p>
        <label for="message">Message Details:</label><br>
        <textarea id="message" name="usermessage" rows="5"
cols="40" placeholder="Type your notes here..."></textarea>
      </p>
      <p>
        <input type="submit" value="Submit Form Data">
        <input type="reset" value="Clear Entries">
      </p>
    </fieldset>
  </form>
</article>

</main>
</td>

<!-- Right Side: Sidebar Column -->
<td width="30%" bgcolor="#f4f4f4">
  <aside id="about">
    <h3>Document References</h3>
    <p>Static pages map data out directly without real-time database
queries or processor overhead.</p>

    <h4>Core Benefits:</h4>
    <ul>
      <li>Instant browser painting</li>
      <li>Low memory footprints</li>
      <li>High data compatibility</li>
    </ul>

    <h4>Required Components:</h4>
    <ol>
      <li>Document Type Definition</li>
      <li>Semantic Wrapper Blocks</li>
      <li>Data Input Control Interfaces</li>
    </ol>
  </aside>
</td>

</tr>
</table>
```

```
<hr>

<!-- Footer Section -->
<footer>
  <center>
    <p>&copy; 2026 Pure HTML Architecture. Assembled completely without style
configurations.</p>
  </center>
</footer>

</body>
</html>
```

5. Compilation / Execution Steps

- Step 1 Create a project folder named exp01/.
- Step 2 Create index.html inside the folder and paste the HTML source code.
- Step 3 Open the exp01/ folder in Visual Studio Code.
- Step 4 Install the Live Server extension if not already installed.
- Step 5 Right-click index.html → select "Open with Live Server".
- Step 6 Browser opens at <http://127.0.0.1:5500/exp01/index.html>.
- Step 7 Verify all sections — header, hero, articles, form, sidebar, footer — render correctly using only default browser styling.

6. Expected Output / Screenshot

The screenshot below shows the StaticWeb page rendered in a Chromium browser, demonstrating pure HTML5 semantic structure — table-based header with navigation, centered hero section, two-column article/sidebar layout, a working inquiry form, and a footer — all without any CSS styling.

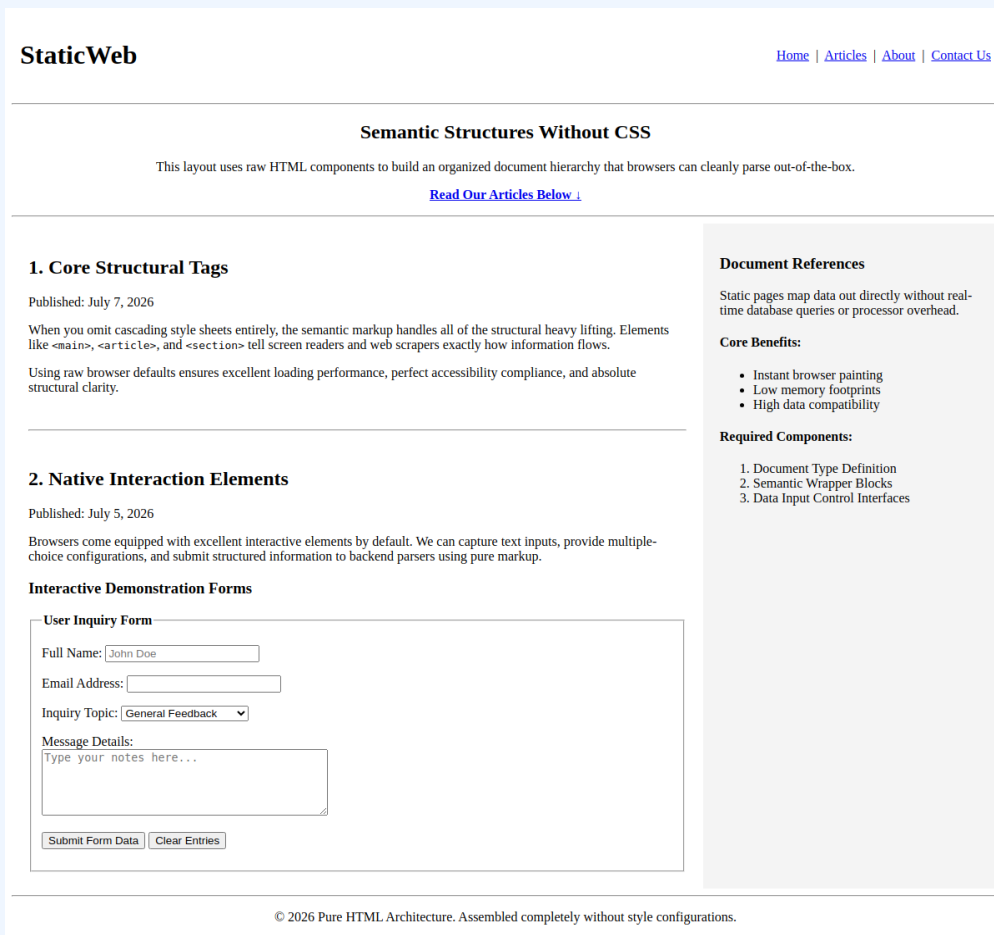


Figure 1: StaticWeb — pure HTML5 semantic webpage rendered in Chromium browser.

7. Result / Conclusion

The experiment was successfully completed. A static webpage was designed and implemented using core HTML5 semantic elements — `<header>`, `<nav>`, `<section>`, `<main>`, `<article>`, `<aside>`, `<form>`, and `<footer>` — along with structural tables and default browser formatting. The page renders a clear, organised document hierarchy entirely through native HTML components, without relying on any CSS, thereby demonstrating the structural role of HTML5 markup independent of presentation styling.

Student Signature

Faculty / Lab In-charge Signature